

In this project, we will build a **monolithic Online Store API**. The system will manage three main areas: **users, products, and orders**, while also following common backend security and performance practices.

The API will use **JWT authentication and Role-Based Access Control (RBAC)** to make sure only authorized users can access certain features. We will also use **Redis rate limiting** to prevent users from sending too many requests and abusing the system.

When a customer places an order, the order and inventory changes will happen inside an **ACID database transaction**. This helps prevent problems such as two customers buying the same product at the same time and accidentally overselling the available stock.

We will also validate incoming data before saving it and use **parameterized queries** to help prevent SQL injection attacks.

To make the database faster, we will add **indexes** to fields that are searched frequently, such as product SKUs and user IDs.

Finally, we will use **structured JSON logs** with a unique **request ID** for each request. This makes it easier to follow a request through the application and find problems when something goes wrong in production.

Please Note — Team & GitHub Instructions

There are **2 teams, with 3 members on each team**. Each team will build its own version of the Online Store API and push the project to the **GitHub organization that has been created for this project**.

Before starting development, each team should discuss and agree on the following:

- **Programming language:** Choose the language your team wants to use, such as Java, Python, JavaScript/TypeScript, Go, C#, etc.
- **Framework:** Choose a suitable framework for your language, if needed.
- **Architecture:** Decide how you want to structure the application. For example, you could use a traditional layered architecture, Clean Architecture, Hexagonal Architecture, or another approach.
- **Database:** Decide which database you will use, such as PostgreSQL or MySQL.
- **Project structure:** Agree on how folders, modules, and responsibilities will be organized.
- **API design:** Agree on how your endpoints, request/response formats, and error handling will work.
- **Git workflow:** Agree on how the team will create branches, make commits, and merge changes.

GitHub Requirements

Each team will have its own project/repository within the GitHub organization.

EVERY TEAM MEMBER MUST MAKE AT LEAST ONE COMMIT TO THEIR TEAM'S PROJECT.

This means you should not have one person build the entire project and simply add the other team members as contributors. Everyone should actively contribute code or another meaningful part of the project.

For example:

1. Team member 1 works on authentication and user management.
2. Team member 2 works on products and inventory.
3. Team member 3 works on orders and transactions.

You should use **Git branches and pull requests** where appropriate so that everyone can contribute without overwriting each other's work.

Your commits should be meaningful and describe what you actually changed. For example:

- Add JWT authentication
- Create product API
- Add order transaction handling
- Add Redis rate limiting
- Add database indexes
- Add request correlation IDs

The goal is not just to get the application working. **The goal is for every team member to practice working on a real collaborative software project using Git and GitHub.**